



Ryan Marsh <ryan.marsh119@gmail.com>

Claude Project!

4 messages

Ryan Marsh <ryan.marsh119@gmail.com>
To: "pxbacher@gmail.com" <pxbacher@gmail.com>

Sat, May 16, 2026 at 10:25 PM

Hey Pete!

Attached you'll find my plan for my Claude project. I'm currently studying to take the Security + 701 exam by the end of June. I'm feeling pretty good about it, but I would like more practice and some way to keep track of the exam objectives I've covered. That's where Claude will come in. In addition to learning how to manage agents, I'll build a tool that will help me study for the exam. I'm going to provide it with resources to pull the questions from, so that I don't need to worry about hallucinations too much with generated questions, but that's a feature that could be implemented later.

I think this would be pretty cool because if I can do this with the topic of Security+, I could potentially use it to learn other topics or study for other certifications I plan to get in the future. The agents' Claude skills should transfer pretty well.

Thanks for giving me a push. I'll work on getting everything set up tomorrow. If you're interested, you can see my progress on my GitHub at: <https://github.com/ryan-marsh119>

V/r

Ryan

 **prompt.txt**
7K

Peter Chrisbacher <pxbacher@gmail.com>
To: Ryan Marsh <ryan.marsh119@gmail.com>

Sun, May 17, 2026 at 9:01 AM

Thanks Ryan

You know the format of that doc you sent me is "Markdown" format, right? If you don't have one already, install a markdown editor/viewer like MarkText and you'll see it like this:

Security+ Study Application Summary

This is a full-stack **general-purpose Security+ SY0-701 study platform** designed for anyone preparing for the CompTIA Security+ exam.

The app turns official CompTIA resources, high-quality YouTube content, free practice tests, and PBQ materials into an interactive learning experience featuring:

- Multiple question types (multiple choice, multi-select, true/false, drag-and-drop, PBQs)
- Practice exams and performance-based question simulators
- Detailed answer keys and explanations
- Progress dashboard with at least 5 milestones tied to official exam domains

The development follows a structured 5-phase process using one Claude Code agent at a time.

Phase 1 Agent: Educational Platform Researcher

Agent Name: EduPlatformResearcher

Starting Prompt:

```
Activate as EduPlatformResearcher.
You are an expert in modern digital learning platforms. Analyze how Quizlet, Anki, Duolingo, and similar
Focus on:
- User learning flows and engagement techniques
- Implementation of different question formats: multiple choice, multi-select, true/false, drag-and-drop,
- Progress tracking, spaced repetition, and dashboards
- How data for these question types is typically structured
Provide PostgreSQL-friendly schema suggestions and best practices tailored for a CompTIA Security+ study
```

Phase 2 Agent: SecurityPlusContentArchitect

Agent Name: SecurityPlusContentArchitect

Starting Prompt:

```
Activate as SecurityPlusContentArchitect.
You are a certified CompTIA Security+ instructor and curriculum designer.
You will be provided with official CompTIA SY0-701 objectives, PDFs, YouTube links, free practice test we
Your tasks:
1. Analyze all provided materials and map them to the official SY0-701 domains and objectives.
2. Identify coverage gaps.
```

Your approach is different from the one that I take in my daily development work with Claude Code - YOU'RE GOING TO TEACH ME STUFF I DON'T CURRENTLY USE.

Some questions

- Do you already have a dev environment setup on your box? Are all the tools you need already installed, configured and tested? If not, add setup of dev environment to your plan - have Claude help or at least plan that.
- I like that you mention git. Local repo only? Offline backup? Repo already setup?
- Will you be using Claude code in a terminal or within VS Code? Or both? (I recommend you try both!) terminal mode has a plan mode (which it looks like you already used for this plan, tho I suspect in an interactive browser session) that I find extremely useful in my work. You should check it out.
- Add documentation production (a manual or similar, including anything you think might be important, but I would think stuff on system requirements, dependencies, OS, install and setup, configuration, file locations, operation, stuff like that.

The above is from me re a read of your plan.

I then consulted with Claude on your plan (of course!) - here's our interaction:

I asked my nephew to put together a plan for learning Claude Code, and he came back with this. I don't want to overwhelm him with additions, but I also want to make sure that this plan will give him a significant taste of Claude code capabilities for someone who is currently a developer upskilling himself to become a strongly proficient AI-literate developer. In case it's not clear, I want my nephew to have the skills and experience needed to be a highly in demand developer within the next five year time frame. What additional suggestions can I make for him to add to this plan, again without overwhelming or discouraging him.

Your nephew's plan is genuinely solid — he's already doing the most important thing, which is *building something real* instead of watching tutorials. That's the right instinct. The plan does have a gap though: it teaches him to use Claude Code as a really smart code generator, but not as an agentic collaborator. Those are different skills, and the second one is what'll make him valuable in five years.

Here are the additions I'd suggest, ordered by impact-to-effort. Pick maybe 3-4 — adding all of them would absolutely overwhelm him.

The high-leverage ones:

1. **Add a `CLAUDE.md` file from day one.** He mentions it once in Phase 4 rules, but he should actually build one and iterate on it across all phases. This is the single most important Claude Code skill — learning what context to persist, how to write instructions that survive across sessions, when to update it. Have him treat it like a living document.
2. **Use sub-agents and the Task tool deliberately.** His plan has "phases" but they're really just sequential prompts. Real Claude Code work involves spawning parallel sub-agents for research, exploration, or verification while the main agent keeps building. He could have a verification sub-agent running alongside Phase 4 that double-checks each migration, for example. This is a skill most devs haven't developed yet.
3. **Add a "plan mode" discipline.** Before each phase, he should explicitly use plan mode (Shift+Tab twice) and review the plan before letting Claude execute. His Phase 4 rules hint at this but don't name it. Learning when to plan vs. when to just-do-it is a real skill.
4. **Build an MCP server, even a tiny one.** This is the thing that separates devs who *use* Claude Code from devs who *extend* it. He could build a small MCP server that exposes the CompTIA objectives as a queryable resource, or one that hits a question bank API. Even a 50-line MCP server teaches him how the protocol works, and MCP fluency is going to be table stakes for AI-literate devs.

The medium ones:

5. **Add a Phase 6: "Ship it."** Deploy to actual infrastructure (Railway, Fly.io, whatever). Have Claude Code handle the deployment, debug production issues, write the CI/CD. Production debugging with Claude is a completely different skill from greenfield building.
6. **Hooks.** Claude Code supports hooks (pre-tool-use, post-tool-use, etc.). Have him set up one or two — maybe a hook that runs tests after every code change, or one that blocks edits to certain files. Small thing, but it teaches him to think about Claude Code as a programmable system, not just a chat interface.
7. **Git worktrees for parallel work.** Once he's comfortable, have him try running two Claude Code instances on different worktrees of the same repo simultaneously — one on frontend, one on backend. This is a workflow pattern that'll be increasingly common.

The "if he's hungry" one:

8. **Evals.** After Phase 5's content review, have him build a small eval harness that programmatically tests whether Claude can correctly answer the Security+ questions he extracted. This teaches eval-driven development, which is a skill almost no traditional developers have yet and which is going to be enormous.

One thing I'd gently push back on: his phase structure is very waterfall-y ("one Claude Code agent at a time"). Real Claude Code work is messier and more iterative — you bounce between phases, you discover the schema is wrong while building the frontend, you refactor. If you can plant the seed that the phases are a starting scaffold rather than a strict gate system, he'll learn faster.

My honest pick of the four to suggest: **CLAUDE.md discipline, plan mode, build a small MCP server, and ship it to production.** That covers context engineering, agentic planning, extending the platform, and real-world ops — which together give him a genuine taste of what makes someone an AI-literate developer rather than just an AI-assisted one.

ALSO FYI:

Does Claude Code only refer to the implementation used in a terminal?

Good question, and no — Claude Code has expanded beyond just the terminal. Let me check the current state of things since this space moves fast. Short answer: no, it's not terminal-only anymore. The terminal CLI is still the original and probably most-used surface, but Claude Code now runs in quite a few places:

- **Terminal/CLI** — the original `claude` command

- **Desktop app** — Anthropic released a fully redesigned Claude Code desktop application for macOS and Windows on April 14, 2026, which supports parallel sessions with Git isolation via worktrees
- **IDE extensions** — VS Code, Cursor, and the JetBrains family (IntelliJ, PyCharm, WebStorm, GoLand, etc.)
- **Web** — a cloud-based version that runs on Anthropic-managed VMs, integrated directly into the desktop app via a Code button next to Chat
- **Slack** — for team workflows
- **CI/CD** — GitHub Actions and GitLab integrations for automated work
- **Mobile** — iOS in preview for monitoring sessions on the go

The interesting part for your nephew: the real power isn't just in having options — it's in the seamless ability to move between them. Start a task on your terminal, monitor it from your phone, review it in your IDE, and automate the whole thing in GitHub Actions.

For his learning plan, I'd still recommend he start in the terminal — it's where the agentic behavior is most visible and where he'll build the strongest mental model of what Claude Code is actually doing. The IDE extensions and desktop app abstract some of that away, which is great for productivity but less great for learning. Once he's comfortable, branching out to the VS Code extension or the desktop app's parallel sessions feature would be a natural next step (and ties directly into the git worktrees suggestion from earlier).

Looks good and I like the idea. Please refine to add any of the above you don't already have set up, then send back your revised plan.

IMO, you should have this whole thing up and fine tuned within a week (probably as soon as a few DAYS). I'm very curious to see how long it takes you.

What's your venmo?

-Pete

Pete Chrisbacher

[Quoted text hidden]

Peter Chrisbacher <pxbacher@gmail.com>
To: Ryan Marsh <ryan.marsh119@gmail.com>

Sun, May 17, 2026 at 9:06 AM

I also asked (for my own benefit)

What is a MCP server?

An MCP server is a small program that exposes tools, data, or actions to an AI model through a standard protocol — **Model Context Protocol**, which Anthropic open-sourced in late 2024.

The easiest way to think about it: MCP is to AI agents what USB is to hardware. Before USB, every device needed its own custom connector. Before MCP, every AI tool integration was a one-off. MCP standardizes the plug, so any MCP-compatible client (Claude Code, Claude Desktop, Cursor, etc.) can talk to any MCP server without custom glue code.

What an MCP server actually does:

It exposes three kinds of things to the AI:

- **Tools** — functions the AI can call (e.g., "create a Jira ticket", "query my database", "fetch the latest stock price")
- **Resources** — data the AI can read (e.g., files, API responses, documents)
- **Prompts** — reusable prompt templates

Concrete example: Say your nephew builds an MCP server for his Security+ app that exposes a `get_objectives` tool returning the official CompTIA SY0-701 domain list, and a `search_questions` tool that queries his question bank. Now any Claude Code session — his, yours, anyone's — can connect to that server and Claude can pull live exam data into its work, without anyone hardcoding the data into prompts.

Why it matters for him specifically:

1. Companies are racing to wrap their internal tools in MCP servers right now. Devs who know how to build them are already in demand.

2. Building one is genuinely easy — a basic Python or TypeScript MCP server is maybe 50-100 lines. The SDK does most of the work.
3. Once he understands MCP, he understands how to *extend* AI agents rather than just use them, which is the leap from AI-assisted to AI-literate.

How it connects to Claude Code: Claude Code can connect to MCP servers and use their tools automatically. So if he writes an MCP server that exposes his Django models as queryable tools, his next Claude Code session can directly interact with his app's data instead of him having to paste schemas into the chat.

Anthropic's docs at modelcontextprotocol.io have a quickstart that gets you running in about 15 minutes. That'd be a great Saturday-afternoon project for him once he's mid-way through his plan.

Pete Chrisbacher

[Quoted text hidden]

Ryan Marsh <ryan.marsh119@gmail.com>
To: Peter Chrisbacher <pxbacher@gmail.com>

Sun, May 17, 2026 at 11:40 AM

Good morning!

To answer your questions:

1. Yes, I currently have VSCode, Python, Postgresql, and Docker already on my laptop, so implementation should be pretty straightforward
2. I'll be posting it on my github, so a remote repo. I'll use Claude to help me set it up
3. I'll definitely be using both, but I do find myself using the terminal more and more, especially with my job. The more I use it, the easier it is to log in to a database for a quick query or check files in a container rather than opening up ANOTHER application and waiting for it to load. But, vscode is definitely useful for git diff or settling merge conflicts.
4. Great idea. In my head, I planned on "docker-izing" the project once I had it working locally, but I didn't add that to the plan. Documentation and a thorough README.md would be a great addition

I like the plan additions of "CLAUDE.md discipline, plan mode, build a small MCP server, and ship it to production". I originally used Grok to flesh out this plan, but it seems that using one AI to assess another AI's capabilities didn't give me a complete picture. I'll sign up today and flesh out the plan more WITH Claude, then send it back to you. You can expect it by the end of today!

Quick question: do you currently use the \$20/month plan yourself, and how often do you reach its limits? One of my concerns was reaching Claude's token limit while using it, so I designed the plan for a more single-agent-at-a-time approach. I've never actually used Claude, so I don't know how fast it can actually deliver something like this. I planned on this taking me the whole month, but maybe it is reasonable to knock this out in a week! I don't know!

My Venmo is @Sarah-Marsh-401. Thanks again, I'm excited about this project!

V/r

Ryan

[Quoted text hidden]